Pacgate AI - 双 AIPC 部署手册

> 在每台机器上克隆仓库，运行相同的安装步骤，两台机器即可完全运行 deer-flow 研究与 qm 协作。
> 版本 0.1.28 - 2026-10-13 英文版本：AIPC-DEPLOYMENT-HANDBOOK.md 前置条件：Docker Desktop、Ollama、Node.js 24+。`install.ps1` 会拉取 `ollama-models.txt` 中列出的模型。

⚠ 重要发现（2026-09-02）— 部署 AIPC #2 前请先阅读

以下问题是在 AIPC #1 试点期间发现的，已在本仓库中修复。 AIPC #2 必须拉取更新后的代码（见 Stage 1），以获得这些修复。

> 2026-11-14 更正（结论不变，证据重写）。 本段原称两个仓库「已不再一致」并给出具体数字，但那些数字已过期，且与本文档第 85 行的说法互相矛盾（那里说两者均可克隆）。结论仍然正确：请克隆 `JZKK720/pacgate-ai-pr`。
>
> 2026-11-14 实测：`pacgate-ai/pacgate-ai-pr` 落后 44 个提交，其 compose 仍固定 0.2.32，因此克隆镜像得到的是上一个版本，而不是 0.2.33/0.2.34。原称镜像「缺少 workflow 接线修复（`b7fc540`、`039afdc`）、只会提供 10 个内置工作流」：该说法已不成立。这两个提交都是镜像 `main` 的祖先，`/app/workflows` 挂载存在，完整工作流库正常提供。不要把这条旧结论继续转述。
>
> 原段落留下的教训仍然有效：本文档曾在没有任何可失败检查的情况下断言两仓库同步。现在改为实测，而实测数字变化了、结论却没有变化。这正是需要重新读取而不是转述的 原因。
>
> 命名空间方面的差异没有变化：向哪个仓库推送标签，决定镜像发布到哪个 GHCR 命名空间。见

1. deer-flow 代理无法查询 pacgate 的法律数据库。 根本原因：没有工具接入 pacgate-api 的 `/api/kb/search`（RAG）或 `/api/search`（法律连接器），且记忆适配器静默回退到本地文件（`PacgateMemoryStorage` 需要 `PACGATE_MATTER_ID`）。 修复：新增 `pacgate-mcp` 服务（FastMCP），向 deer-flow 暴露 `pacgate_kb_search`、`pacgate_connector_search`、`pacgate_list_connectors`。 已在 `deer-flow-extensions-config.json` 中与 openviking 并列注册。
2. openviking MCP 内置了错误的 API 密钥。 `deer-flow-extensions-config.json` 使用了应用密钥（`OPENVIKING_API_KEY`），但 openviking 的 `root_api_key` 是 `OPENVIKING_ROOT_API_KEY`。错误的密钥导致 openviking 返回 401，进而回滚了 整个 MCP 工具加载（deer-flow 使用 `asyncio.gather`），因此没有任何 MCP 工具出现。修复： 使用 `OPENVIKING_ROOT_API_KEY`（模板现已使用 `${OPENVIKING_ROOT_API_KEY}`）。
3. 配置变更优先使用 `docker compose restart deer-flow`，而不是 `--force-recreate`。 `.deer-flow`（SQLite 数据库、管理员用户、线程、`.jwt_secret`）现已绑定挂载到 `./data/deer-flow:/app/backend/.deer-flow`，`compose.prod.yaml` 与 `compose.bundle.yaml` 两个 lane 都是如此，因此重建容器不再丢失这些数据。 实测 2026-10-28：在开发机上对 deer-flow 执行 `--force-recreate`，数据库、 `10/21` 的 `.jwt_secret` 与用户线程均完好保留。 本条目此前写作"重建会丢失数据库"，该理由已过时。这条建议本身仍然成立， 但理由不同：重建会重新执行启动流程并中断正在运行的 run；而 `patches/*.py` 与 `deer-flow-config.yaml` 只在进程启动时读取，`restart` 已足以生效。仅在 compose 文件本身变化时（新增挂载、环境变量或镜像）才需要重建。
4. QM 登录需要邮件传输，而不是 Outlook SMTP。 旧的 SMTP 路径 （`smtp.office365.com` + 应用密码）已失效——微软已于 2025 年 9 月停用 Exchange Online 的基本身份验证 / 应用密码。`qm check` 失败并返回 `535 5.7.139 Authentication unsuccessful`。修复： qm 的 auth 代理现在使用 Resend 传输（`AUTH_EMAIL_TRANSPORT=resend`）。你必须在 `deploy/qm-pacgate/.env` 中提供 `RESEND_API_KEY`（见 Stage 4）。此条最初称 Resend 是唯一选项，Stage 4 的双传输设置已取代——试点用 Mailpit，无需密钥。

5. qm web-ui 无法自行认证。其服务器（`/app/server/index.ts`）设置 `AUTH_MODE = COOKIE_AUTH ? "dev" : "portal"`。因为设置了 `CORE_SIGNING_SECRET`，它处于 portal 模式，需要 portal 签发的身份令牌。没有无需密钥的方式直接访问 web-ui——你必须运行 `portal+auth`（Resend）或外部 OIDC 提供方。`ADMIN_GRANTS` 是授权种子，不是登录方式。
6. `pacgate-ai` 账号向 `JZKK720/pacgate-ai-pr` 推送被阻止（403，需要 2FA 授权）。变通方案：`pacgate-ai` 账号可以创建 fork 并推送到那里。但 fork `pacgate-ai/pacgate-ai-pr` 是一个只读镜像，会滞后：2026-11-14 实测落后 44 个提交、仍固定 0.2.32。所以「推送到 fork 就等于完成同步」并不成立，见 Stage 1。

架构：两台相同的机器

两台 AIPC 都运行完整的栈：

```
■■ AIPC ■■■
  nginx :8089  -> pacgate-api :8080■Rust ■■■ API■
                -> deer-flow  :8001■■■■■■■■
  Postgres :5432■■■■■■■■■■■
  OpenViking :1933■■■■■■■■■MCP■
  qm :8182■■■■■■■■■■■ `qm up` ■■■
  Ollama :11434■■■■■■■GPU/NPU■
```

每台机器都是自包含且独立可运行的。任一机器上的律师都可以使用研究模式（deer-flow，位于 `http://localhost:8089/research/`）和协作模式（qm，位于 `http://localhost:8182`），无需依赖另一台机器。

如果之后希望在两台机器之间共享事项数据，请用私有网格（Tailscale 或 WireGuard）连接它们，并决定同步或单一权威模型。这是试点后的决策，不是部署前置条件。

开始前需要准备什么

- 源码仓库访问权限：请使用 `JZKK720/pacgate-ai-pr`（公开；单纯克隆无需认证）。不要克隆 `pacgate-ai/pacgate-ai-pr` 镜像：它落后 44 个提交，compose 仍固定 0.2.32，克隆它部署的是上一个版本（见 Stage 1）。仅当需要推送时才需 PAT 或 `gh auth login`。
- 两台 AIPC 上都运行 Docker Desktop
- 两台 AIPC 上都运行 Ollama（`install.ps1` 会拉取它需要的模型）
- 如果使用带 cloud 标签的 deepseek 模型，每台 AIPC 上完成 `ollama signin`
- 两台 AIPC 上都安装 Node.js 24+（供 qm 使用）
- PowerShell 7（`pwsh`）可选 —— 安装脚本在系统自带的 PowerShell 5.1 上同样可以运行。（在 2026-10-11 之前这一说法并不成立，且有两处问题：其一，只要附加了重定向，PS 5.1 就会把任何写入 stderr 的原生命令变成终止性错误，而 `docker compose up -d` 每次运行都会把进度写入 stderr，导致安装脚本在引导租户和管理员之前就中断；其二，管理员注册调用使用了 PS 7 专有的 `-SkipHttpErrorCheck` 参数，5.1 会在请求发出之前就拒绝它，于是每一次 API 调用都静默失败。现在 `install.ps1`、`setup-qm.ps1` 和 `smoke-deer-flow-outputs.ps1` 中所有写 stderr 的调用都经由 `Invoke-WithNativeStderr`，所有 API 调用都经由 `Invoke-ApiRequest`。详见 `deploy/DEFECT-ps51-native-stderr.md`。）
- 无需 `docker login ghcr.io`——Pacgate 运行时镜像以公开 GHCR 包发布（见 Stage 0）。

Stage 0：运行时镜像（开发机，已完成）

运行时已发布到 GHCR，AIPC 上无需重建。

当前版本（应拉取的版本）：

镜像	状态
ghcr.io/jzkk720/pacgate-api:0.2.34	脱敏修复版本。 修正了校验器，使其不再在文档仍含身份证号、统一社会信用代码、律师执业证号、银行账号、港澳通行证、车牌、微信号、纳税人识别号时判为通过（详见下文）。同时保留案件工作区汇总视图，并新增两个补救接口： POST /api/documents/:id/sanitize-rescan 与 PATCH /api/documents/:id/sanitize-reset 。
ghcr.io/jzkk720/pacgate-mcp:0.2.34	向 deer-flow 暴露 26 个 MCP 工具，其中 pacgate_rescan_residue 与 pacgate_reset_sanitization 供补救流程使用。注意 server.py 以 bind-mount 挂入容器，因此 git pull 加 pacgate-mcp 重启即可获得这些工具，无需新镜像。
ghcr.io/jzkk720/deer-flow-pacgate:0.2.34	本版本未变更。
ghcr.io/jzkk720/deer-flow-frontend-pacgate:0.2.34	本版本未变更。
ghcr.io/jzkk720/ocr-service:0.2.34	PaddleOCR 抽取服务；自 0.1.16 起为一等镜像。自 0.1.28 起，其 PP-OCRv4 权重已 内置于镜像 ，抽取无需外部模型站点，可离线运行。
ghcr.io/volcengine/openviking@sha256:46f9e34c…	在 compose.prod.yaml 中按摘要固定。上游公开镜像。

> 0.2.14 是正确性修复，不是功能版本。客户扫描真实扫描件时返回 `verdict: pass`、13 处替换，而八类标识符原样保留在被检索的文本里；该文档还带有 `sanitized` 标记，下游因此不再复核。账本记录的原因是 `"noise replay: 1 of 1 surviving matches dropped by the same suppressor the redaction used"`：校验器重放了脱敏器自己的抑制器，抑制器在入口漏掉的，出口必然同样漏掉。
>
> 现在校验器绝不允许该抑制器触碰规则匹配。修复只对后续生效，因此另有两项能力：`pacgate_rescan_residue` 按库中实际存储的分块重扫（仅规则层，不加载模型）并指出残留内容；`pacgate_reset_sanitization` 把 `sanitized` 文档退回 `pending`，使重跑是真正重写而非空操作。在完成一次重扫之前，不要把该机器的 RAG 结果当作干净数据使用。
>
> 命名空间与版本于 2026-09-22 更正。此表此前列出 `ghcr.io/pacgate-ai/*` 的 0.1.0/0.1.3。`pacgate-ai` 为遗留镜像，实际命名空间为 `jzkk720`，发布自 plan 016 起已迁移。旧表中的 `pacgate-ai/...-frontend-pacgate:0.1.0` 行标注“已发布”，实际返回 404——从未存在。五个 `jzkk720/*` 镜像按当前固定值（0.1.28，见上表）匿名拉取均返回 HTTP 200。

历史发布表（保留以追溯，已被取代）：

镜像	状态
ghcr.io/pacgate-ai/pacgate-api:0.1.3	当时已发布。修复 0.1.1 的容器网络 bug。
ghcr.io/pacgate-ai/pacgate-mcp:0.1.3	当时已发布。
ghcr.io/pacgate-ai/deer-flow-pacgate:0.1.3	当时已发布。

镜像	状态
ghcr.io/pacgate-ai/deer-flow-frontend-pacgate:0.1.0	从未发布——返回 404。 请勿使用。

所有 Pacgate 包必须在 GHCR 上设置为公开可见，以便 AIPC 无需注册表凭据即可拉取。 上线前验证：

```
# ■■■■ docker login ■■■ HTTP 200■401/403 ■■■■■■■■■
# 404 ■■■■■■■■■■■■■■—■■■ compose.prod.yaml ■■■■■■
# ■■■■■■■■■■■
#
# ■■■■ Accept ■—■■■■■■■■■■ 404■■■■ 200■■
#
# ■■■■■■ compose.prod.yaml ■■■■■■■■■■■■■■■■■■■■
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
$acc = "application/vnd.oci.image.index.v1+json,application/vnd.docker.distribution.manifest.list.v2+json,ap
$compose = "deploy/client-bundle/compose.prod.yaml"
$pins = Select-String -Path $compose -Pattern "image:\s*(ghcr\.io/[^\s]+)" -AllMatches |
        ForEach-Object { $_.Matches } | ForEach-Object { $_.Groups[1].Value } |
        Where-Object { $_ -notmatch "openviking" } | Sort-Object -Unique

foreach ($pin in $pins) {
  $repo = $pin -replace "^ghcr\.io/", ""            # owner/name:tag
  $name = ($repo -split ":")[0]                       # owner/name
  $tag  = ($repo -split ":")[1]
  $t = (Invoke-RestMethod "https://ghcr.io/token?scope=repository:$name`:pull").token
  $code = (Invoke-WebRequest -Uri "https://ghcr.io/v2/$name/manifests/$tag" `
            -Headers @{Authorization="Bearer $t"; Accept=$acc} `
            -Method Head -UseBasicParsing).StatusCode
  "{0,-58} => HTTP {1}" -f $pin, $code
}
```

切换可见性（GitHub Web UI——个人账号的 API 路由返回 404）：GitHub → 你的个人资料 → Packages → `pacgate-api` → Package settings → Visibility → Public → Save。对 `deer-flow-pacgate` 重复此操作。这是安全的：镜像只包含 编译后的二进制文件和 SQL 迁移，所有密钥都在运行时通过 `.env` 注入，且安装程序 已拥有对相同代码的完整源码访问权限。

清理历史镜像版本

六个包累积了约 210 个标签，绝大多数早已被取代。客户端安装或回滚只需要最新的三个发布版本。

```
# ■■■■■■■■■■■■■■■■■■■■■■
.\scripts\prune-ghcr-versions.ps1

# ■■■■■■■■■■ read:packages + delete:packages ■■■■■■■
$env:GHCR_PRUNE_TOKEN = '<token>'
.\scripts\prune-ghcr-versions.ps1 -Execute
```

有三点比表面看起来复杂，脚本正是为解决它们而存在：

- registry API 无法删除 GHCR 清单。`DELETE /v2/<repo>/manifests/<digest>` 对任何凭据 都返回 405。删除必须走 GitHub Packages REST API，且令牌需同时具备 `read:packages` 与 `delete:packages`；普通 `repo` 权限的令牌会得到 `403`。注意：`ghcr.io/token?...:delete` 返回 `200` 不能证明有删除权限——它无论令牌能否删除都会返回 200。
- GitHub Actions 工作流同样做不到，而且这是结构性的。`jzkk720` 是用户账号，不是组织，因此它拥有的包位于 `/user/packages/...` 下。自动的 `GITHUB_TOKEN` 是安装令牌：它永远无法以用户身份行事，其 `/repos/{owner}/{repo}/packages` 路径只能看到已关联到该仓库的包。2026-11-03 在 Actions 中以 `permissions: packages: write` 实测：仓库包列表、单个包、包版本、以及 `/orgs/jzkk720/...` 变体全部返回 404。本手册早先版本的说明曾建议用工作流，因它永远不可能成功而删除。唯一可行路径是 具备 `delete:packages` 的用户级令牌。

- `latest` 可能与某个发布标签共享同一摘要。 删除旧标签背后的版本会把 `latest` 一起删掉。计划会保护所有被保留标签或 `latest` 指向的摘要，并明确列出而不删除。
- 一个摘要可能挂多个标签。 实测 `0.2.13` 与遗留的 `tmp-del` 指向同一清单，计划会将它们合并为一次删除。若分开列出，第二次 DELETE 会返回 404，把一次完全成功的执行 报告成失败。

删除不可逆，因此脚本默认试运行，需要 `-Execute` 加手动确认，并由 `scripts/test-prune-ghcr-versions.ps1`（离线夹具、无需凭据）覆盖，断言它不会删除本应保留的版本，也不会误报自身结果。

请至少保留你要回滚到的那个版本：运行 `0.2.16` 的客户端需要退回时，`0.2.15` 应当仍可拉取。

仅当 Rust 源码变更时，才在开发机上重建并推送。标签值应从 compose 固定值中读取，而不要手写版本号——此处硬编码的标签已两次过期，其中一次还把健康系统误报为故障：

```
cd c:\Users\cubecloud-io\github-pr\pacgate-ai-pr
# ■■■■■■■■■■■■■jzkk720■■■■■■■■ 'pacgate-ai/pacgate-api'■
# ■ compose.prod.yaml ■■■■■■■■■■$tag ■■■■■■■■■■■■/■■
# ■■■■■■■■■
$tag = (Select-String -Path deploy/client-bundle/compose.prod.yaml `
        -Pattern 'jzkk720/pacgate-api:(\S+)').Matches.Groups[1].Value
if (-not $tag) { throw 'could not read the image tag from compose.prod.yaml' }
docker build -t ghcr.io/jzkk720/pacgate-api:$tag -f pacgate-ai/Dockerfile ./pacgate-ai
docker push  ghcr.io/jzkk720/pacgate-api:$tag
```

实践中建议使用 `build-ghcr.yml` 工作流，以保证四个镜像版本一致——参见 `deploy/README-BUILD.md` 与 `plans/012-master-release-namespace.md`。

不要在 AIPC 上重建——试点运行已发布的摘要。

> 端口说明： 栈将 nginx 绑定到主机端口 `8089`（`deploy/client-bundle/compose.prod.yaml` 中提交的值）。如果该端口在机器上已被占用， 请编辑 `deploy/client-bundle/compose.prod.yaml` 中 `nginx` 的 `ports:` 条目， 并在下方所有验证 URL 中使用新端口。

Stage 1：在每台 AIPC 上克隆仓库

在两台机器上：

```
cd C:\
git clone https://github.com/JZKK720/pacgate-ai-pr.git
cd pacgate-ai-pr
git remote -v   # origin ■■■ JZKK720/pacgate-ai-pr
```

> AIPC #2 说明（2026-11-14 更正，结论不变）。 请克隆 `JZKK720`。
>
> 原段落称 `pacgate-ai/pacgate-ai-pr` 落后 26 个提交、缺少 `b7fc540` 与 `039afdc`、 只提供 10 个内置工作流。这些数字已过期：2026-11-14 实测落后 44 个提交，且 `b7fc540` / `039afdc` 都已包含在其 `main` 中，`/app/workflows` 挂载存在，完整 工作流库正常提供。真正的问题是它的 compose 仍固定 0.2.32，克隆镜像得到的是上一个版本。上面「内容完全一致」的旧说法在写入时是准确的，分歧发生在之后。这正是文档不应在没有可失败检查的情况下断言两者同步的原因。
>
> 命名空间方面的差异没有变化：向哪个仓库推送标签，决定镜像发布到哪个 GHCR 命名空间。见

仓库为公开，克隆无需凭据；仅当需要推送时才使用个人访问令牌或 GitHub CLI（`gh auth login`）。

Stage 2：部署核心栈（两台机器，步骤相同）

在每台 AIPC 上运行这些步骤。Docker Compose 栈会启动 pacgate-api、Postgres、nginx 和 deer-flow。

```
cd C:\pacgate-ai-pr\deploy\client-bundle
```

```
copy .env.example .env
notepad .env
```

填写这些值：

```
PACGATE_DB_PASSWORD=<[redacted]>
PACGATE_JWT_SECRET=<[redacted]>
PACGATE_TENANT_ID=pacgate-law
OPENVIKING_ROOT_API_KEY=<[redacted] 32 [redacted]>
OPENVIKING_API_KEY=<[redacted] 32 [redacted]>
```

`OPENVIKING_API_KEY` 是必需的——安装程序会用它渲染 `deer-flow-extensions-config.json`，如果缺失或保留为 `change-me` 则报错停止。

如果需要，生成密钥：

```
# [redacted]16 [redacted]
-join ((1..16) | ForEach-Object { '{0:x}' -f (Get-Random -Maximum 16) })

# JWT [redacted]32 [redacted]
-join ((1..32) | ForEach-Object { '{0:x}' -f (Get-Random -Maximum 16) })

# OpenViking [redacted] 32 [redacted]—[redacted]
-join ((1..32) | ForEach-Object { '{0:x}' -f (Get-Random -Maximum 16) })
```

运行安装程序：

```
.\install.ps1
```

安装程序会拉取（公开、无需登录的）GHCR 镜像，从 `.env` 密钥渲染 `OPENVIKING_CONF_CONTENT` 和 `deer-flow-extensions-config.json`，启动 Docker Compose 栈，并拉取 `ollama-models.txt` 中列出的 Ollama 模型。如果模型已拉取，此步骤很快。

验证核心栈：

```
docker compose -f compose.prod.yaml ps
curl http://localhost:8089/version
curl http://localhost:8089/pacgate/health
```

预期：八个容器全部运行（pacgate-db、pacgate-api、deer-flow、deer-flow-frontend、pacgate-mcp、ocr-service、openviking、nginx），`/version` 返回 `{"version":"0.1.28","revision":"<git sha>"}`，且 `/pacgate/health` 返回 `ok`。

> 不要在 nginx 根路径探测 `/health`。nginx 按设计将 `/` 路由到 deer-flow 前端，`curl http://localhost:8089/health` 会返回前端的 404 页面——看似失败，实为正确路由。`/version` 也在根路径（映射到 API 的 `/build-info`）；只有 `/pacgate/*` 路径会到达 API。

Stage 3：初始化租户并开通账号（两台机器）

> `install.ps1` 现已自动完成（step 6a）。在新版安装上无需手工执行，本节保留用于恢复场景，其中的命令与安装程序实际运行的命令一致。

自 0.1.22 起的账号机制（开通人员前请先阅读）

- `POST /api/auth/register` 仅限首位用户：在全新部署上只创建一个账号（引导管理员），此后一律返回 403。
- 此后的每个账号由管理员通过 `POST /api/auth/users` 创建（Bearer = 管理员令牌）。
- 检索工作区登录 = 邮箱 + 密码；协作（qm）登录 = 一次性邮件链接，受 `AUTH_ALLOWED_EMAILS` 名单控制。

先初始化租户，再由安装程序引导创建管理员：

```
# ■■■■■■■■■slug ■■■ PACGATE_TENANT_ID ■■■
docker exec pacgate-db psql -U pacgate -d pacgate -c "INSERT INTO tenants (name, slug) SELECT 'Default Firm'
```

开通律师账号（管理员账号存在后执行）：

```
$login = Invoke-RestMethod -Uri "http://localhost:8089/pacgate/api/auth/login" -Method Post `
  -Body '{"email":"admin@pacgate-law.com","password":"<admin-password>"}' `
  -ContentType "application/json"
$body = @{email="<attorney-email>"; password="<attorney-password>"; role="attorney"} | ConvertTo-Json
Invoke-RestMethod -Uri "http://localhost:8089/pacgate/api/auth/users" -Method Post `
  -Headers @{Authorization="Bearer $($login.token)"} -Body $body -ContentType "application/json"
```

注册 qm 桥接服务账号（安装程序自动完成，此处保留手工方式用于恢复），
使用相同的管理员开通路由：`email="qm-bridge@pacgate-law.com"`。

协作工作区侧：在 `deploy/client-bundle/qm-pacgate/.env` 中设置
`AUTH_ALLOWED_EMAILS`（逗号分隔）与 `ADMIN_GRANTS=<email>:org_admin`，然后重新运行
`setup-qm.ps1`。

Stage 3.5：验证 OpenViking 记忆服务（两台机器）

OpenViking 是长期记忆通道：deer-flow 和 qm 在那里存储对话上下文，并在后续会话中召回。它作为 compose 栈的一部分启动。

```
curl http://localhost:1933/health
```

预期：`{"status":"ok","healthy":true,...}`。安装程序将 OpenViking 配置（Ollama 嵌入 + VLM）渲染到 `.env` 中的 `OPENVIKING_CONF_CONTENT`，并在首次启动时 初始化服务器的 `ov.conf`。

功能检查（可选，使用 `.env` 中的根密钥）：

```
$key = (Get-Content .env | Select-String '^OPENVIKING_ROOT_API_KEY=').Line.Split('=')[1]
$body = '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'
curl.exe -s -X POST http://localhost:1933/mcp -H "X-API-Key: $key" -H "Content-Type: application/json" -H "A
```

预期：工具列表包含 `find`、`search`、`read`、`remember`。

边界规则：OpenViking 只存储对话上下文（决策、偏好、工作知识）。事项文档和 T1-T4 受控内容保留在 pacgate-api/pacgate-rag 中。

Stage 4：引导 qm（两台机器，步骤相同）

qm 独立于 Docker Compose 栈运行。在核心栈健康后，在每台机器上引导它。

```
cd C:\pacgate-ai-pr\deploy\client-bundle
.\setup-qm.ps1
```

脚本会提示输入：- 管理员工作邮箱（小写）- Pacgate 桥接邮箱：`qm-bridge@pacgate.local` - Pacgate 桥接密码：你在 Stage 3 中注册的那个

脚本会生成签名密钥，在 qm-pacgate 目录中创建 `.env`，用 `qm check` 验证配置，用 `qm sandbox build` 构建沙箱镜像，取得（SHA-256 校验后的）核心容器所需的 静态 docker CLI，并构建 `qm-pacgate-sandbox-local` exec-daemon 包装镜像。

QM 登录需要邮件传输。qm 的 auth 代理发送一次性登录链接。支持两种传输：

- 本地/试点拓扑（Mailpit SMTP 捕获器）：`SMTP_HOST=mailpit SMTP_PORT=1025 AUTH_EMAIL_TRANSPORT=smtp SMTP_TLS=none`。链接落在 Mailpit 收件箱中——试点时 打开 `http://localhost:8025` 领取登录链接即可，不发送真实邮件。
- 生产拓扑（Resend）：`AUTH_EMAIL_TRANSPORT=resend` 并在 `deploy/qm-pacgate/.env` 中设置 `RESEND_API_KEY`，`AUTH_EMAIL_FROM` 使用 Resend 已验证的发件人（Outlook 地址不属于已验证；微软已停用 Exchange Online 的基本身份验证/应用密码，因此原生 Outlook SMTP

不受支持）：

```
RESEND_API_KEY=re_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
AUTH_EMAIL_FROM="PacGate <onboarding@resend.dev>"
```

要么在 Resend 中验证一个真实域名（例如 `pacgate-law.com`），要么暂时使用 Resend 的测试发件人：

```
AUTH_EMAIL_FROM="PacGate <onboarding@resend.dev>"
```

启动 qm：

```
cd C:\pacgate-ai-pr\deploy\client-bundle\qm-pacgate
docker compose -f compose.qm.yaml up -d
```

> 请用 compose，不要用 `qm up`。`@yc-software/qm` CLI 的 docker 生命周期 只有 POSIX 一条路——它的 `which()` 直接执行 `/bin/sh`，在 Windows 上不存在 （2026-10-06 实测：`execFileSync("/bin/sh", …)` → `ENOENT`），所以 `qm up`、`qm down`、`qm status` 在任何 AIPC 上都会报 "docker not found on PATH"。`compose.qm.yaml` 描述的是同一套拓扑（同名容器、同名卷、同一网络），并且 额外承载 Pacgate 补丁：`patch/pi-models.ts` 与 `patch/local-sandbox.ts` bind-mount 覆盖核心源码、静态 docker CLI 与宿主 docker socket 均已挂载、 沙箱包装镜像已接线。这些挂载在每次 `down/up -d` 循环中都保持不变—— 模型路由与沙箱链路在 compose 路径上是持久的。不要对同一目录同时运行 `docker compose -f compose.qm.yaml up -d` 和 `qm up`（两者会争抢同名卷与网络）。

验证 qm：

```
# ■■ http://localhost:8182  (web-ui) — ■■ portal ■■■■
# ■■ http://localhost:8181  (portal) — ■■■■
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ Mailpit■
# ■■■■■■■■■
# ■■■"List available pacgate workflows"
```

> Web-ui 认证现实： qm web-ui（`:8182`）无法自行认证任何人——它必须通过 portal（`:8181`）访问，由 portal 签发身份令牌。如果你直接打开 `:8182`， 会看到 "reached through the portal."。始终通过 `:8181` 访问。
>
> 开发模式替代方案（试点 / 单用户）： 对于本地试点，你可以让 qm 以 dev/cookie 模式运行，而不是 portal。用 `NODE_ENV=development` + `ALLOW_UNAUTHENTICATED_CORE=1` 且不设置 `CORE_SIGNING_SECRET` 重建 `qm-pacgate-core`，并用不设置 `CORE_SIGNING_SECRET` 重建 `qm-pacgate-web-ui`。 然后 `POST /signin` 直接在 `:8182` 用 `{"user":"<principal>"}` 工作，且无需 Resend 密钥。这不是生产正确配置（无认证）——仅用于单用户试点。

Stage 5：验证 deer-flow（两台机器）

deer-flow 有自己独立的登录体系，与 `pacgate-api` 的登录相互独立。
这是「密码不对」报告最常见的单一原因，因此在尝试登录之前请先读这一节。

存在两个独立身份存储，且不共用密码：

界面	URL	身份存储	密码来源
pacgate-api（API，由 install.ps1 预置）	/pacgate/api/auth/login	Postgres users 表	.env → PACGATE_API_PASSWORD
deer-flow（研究界面）	/api/v1/auth/*	SQLite data/deer-flow/data/deerflow.db	由 deer-flow 自身的首次运行设置决定

`PACGATE_API_EMAIL` / `PACGATE_API_PASSWORD` 是 pacgate-api 的主体凭据，不是 deer-flow 的凭据。把它们用于 deer-flow 登录会返回 `401 invalid_credentials` —— 这读起来

与「密码错误」完全一样，但实际是「主体不对」。`install.ps1` 不会创建 deer-flow 管理员；它也无法创建，因为 deer-flow 的管理员是由其自身的 `/initialize` 建立的。

登录 deer-flow：

```
# ■■ http://localhost:8089/research/
# ■■■■■■■■■■■■■■■■ ── ■■■■■ deer-flow ■■■
# ■■■■■■■■■■ deer-flow ■■■■■■■■
```

如果 deer-flow 管理员密码确实丢失，可以在不影响其他任何数据的前提下重建。密码格式为 bcrypt(base64(sha256(password)))，由 `app/gateway/auth/password.py` 写入：

```
# 1. deer-flow ■■■■■■■■■■■■■■■needs_setup=false ■■■■■■■■■■
curl.exe -s http://localhost:8089/api/v1/auth/setup-status

# 2. ■ deer-flow ■■■■■■■■■■■■■■■■■■■■■■■
#    ■■■■■■■■ /data/deer-flow/data/deerflow.db ■■■
#    ■■■■■■ deploy/BENCHMARK-REPORT-2026-10-08.md ── 2026-10-08 ■■■■■■■■
#    ■■■■■■■■■■■■■■■■■■■■■■■■■ → ■■■■■■■■■
```

随后验证研究工作空间可用：

```
# ■■ http://localhost:8089/research/
# ■■■■■■■■■■
# ■■■"Summarize recent force majeure case law in China"
# ■■■■■■■■■■
# ■■■■■■■■■■■■■■
```

一个无需凭据的机械检查：用明知错误的密码登录，必须返回 401 `invalid_credentials` —— 而不是 403（意味着 Origin 门控配置有误，浏览器会被挡在外面），也不是 5xx（认证服务异常）。

Stage 5.5：全链路操作 — deer-flow ↔ QM ↔ OpenViking

两个工作空间共享一个 OpenViking 长期记忆通道和一个 pacgate-api 元数据存储。
本节说明两个系统如何相互通信，以及如何端到端验证该链路。

拓扑（2026-09-04 已验证）

```
pacgate-ai-bundle_default  (Docker Compose ■■)
■■■ openviking :1933   ← ■■■■■MCP■find/search/read/remember■
■■■ pacgate-api :8080  ← ■■■ API■■■/■■■/■■■■
■■■ pacgate-mcp :8000  ← FastMCP ■■■■■ pacgate KB/■■■■■
■■■ laya :8000         ← ■■■■■■■■■■■■ profile■
■■■ deer-flow :8001    ← ■■■■■■■■■ openviking + pacgate-mcp■
■■■ nginx :8089        ← ■■■deer-flow ■■ + /pacgate/ API■

qm-pacgate  (qm up ■■)
■■■ qm-pacgate-core    ← ■■■■■■■■
■■■ qm-pacgate-web-ui  ← ■■■■■ UI■:8182■
■■■ qm-pacgate-pg      ← qm Postgres

■■■qm-pacgate-core ■■■■ pacgate-ai-bundle_default■■■■■■■■■
openviking:1933■pacgate-api:8080 ■ host.docker.internal:11434■Ollama■■
```

链路如何工作

1. deer-flow → OpenViking：`deer-flow-extensions-config.json` 将 `openviking` 注册为 `http://openviking:1933/mcp` 的 HTTP MCP 服务器，使用根 API 密钥（`OPENVIKING_ROOT_API_KEY`）。研究运行在那里存储和召回对话上下文。
2. deer-flow → pacgate：`pacgate-mcp`（FastMCP）向 deer-flow 暴露 `pacgate_kb_search` / `pacgate_connector_search` / `pacgate_list_connectors`，由 pacgate-api 的 RAG + 法律连接器支撑。

自 0.2.10 起，同一服务还暴露三个条款工具：`pacgate_clause_risk`、`pacgate_clause_flags`、`pacgate_route_language`，转发给可选的 `laya` 服务。 laya 是本地 System 1 判定引擎：对单个条款提出定型问题（风险高低、有哪些风险标记、偏向哪一方），一次前向推理即返回校准后的概率，而不是生成式文本。 它不需要 API 密钥，也不发起任何云端调用——这正是选择它而不用按次计费的判定 API 的原因。

它默认关闭。两个检查点合计约 1.45 GB，运行它的机器与不运行它的机器功能都 完整：三个工具会返回 `available: false` 并附带原因，其余工具完全不受影响。 自 0.2.13 起，检查点已打包进镜像，因此启用只需一条命令：`docker compose --profile laya up -d laya`。无需预置权重，运行时也不下载任何内容 ——权重在构建镜像时校验 SHA-256，服务以 `HF_HUB_OFFLINE=1` 运行。 请为它预留 4.4 GiB 主机内存：这是两个检查点均加载后的实测空闲占用，因为每个 检查点各持有一份 fp32 副本。这是主机内存而非显存；在 16 GiB 且已驻留大模型的 机器上，建议先确认可用内存。

两个检查点都需要。laya 按文字体系路由：拉丁文本走 `english`，中文走 `multilingual`；缺少任一检查点，该语言的请求会返回 HTTP 500。实测英文条款上 `english` 检查点也更准确。 3. QM → OpenViking：QM 核心有 `OPENVIKING_URL=http://openviking:1933` 和 `OPENVIKING_API_KEY`（根密钥）。`pacgate-qm` 沙箱工具（`ov-remember` / `ov-search` / `ov-read`）在代理沙箱内通过 `http://host.docker.internal:1933` 调用 OpenViking，并带 `OPENVIKING_ACCOUNT=pacgate-law` + `OPENVIKING_USER`。 4. QM → pacgate：`pacgate-qm` 沙箱工具登录 pacgate-api（`PACGATE_API_EMAIL` / `PACGATE_API_PASSWORD`）以发现工作流、将 QM 作用域绑定到事项、读写事项记忆， 并执行工作流。

验证全链路

```
# 1. OpenViking MCP ■■■■■■ .env ■■■■■■■
$key = (Get-Content .env | Select-String '^OPENVIKING_ROOT_API_KEY=').Line.Split('=')[1]
$body = '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'
curl.exe -s -X POST http://localhost:1933/mcp -H "X-API-Key: $key" -H "Content-Type: application/json" -H "A
# ■■■■■■ find/search/read/remember

# 2. QM ■■■■■ openviking + pacgate-api + Ollama
docker exec qm-pacgate-core sh -c "curl -s -o /dev/null -w 'openviking:%{http_code}\n' http://openviking:193

# 3. QM ■■■■■■dev/cookie ■■■■
$body = '{"user":"admin@pacgate-law.com"}'
curl.exe -s -X POST http://localhost:8182/signin -H "Content-Type: application/json" -d $body
# ■■■■{"ok":true,"user":"admin@pacgate-law.com"}
```

QM ↔ bundle 网络加入（幂等）

QM 核心已加入 bundle 网络，以便按名称解析 `openviking` 和 `pacgate-api`。 如果 `qm up`/`qm down` 循环丢失了加入，请重新应用：

```
docker network connect pacgate-ai-bundle_default qm-pacgate-core
```

> 注意： QM 有意不是 bundle 中的 Docker Compose 服务。它由 `@yc-software/qm` CLI（`qm up` / `qm down`）管理，有自己的生命周期。 网络加入是唯一的耦合——请保持这样。不要将 QM 重写为 compose 服务； 那会与 qm CLI 冲突，并在每次 `qm up` 时重建容器。

Stage 6：冒烟测试清单（两台机器）

在每台 AIPC 上独立运行此清单。

核心栈

- [ ] `docker compose -f compose.prod.yaml ps` 显示 8 个服务运行（含 openviking、pacgate-mcp、ocr-service、deer-flow-frontend）

- [ ] `curl http://localhost:8089/version` 返回发布版本与 revision JSON；`curl http://localhost:8089/pacgate/health` 返回 `ok`
- [ ] `curl http://localhost:1933/health` 返回健康 JSON
- [ ] Postgres 有 `pacgate-law` 租户
- [ ] 管理员用户可以在 `http://localhost:8089/api/auth/login` 登录
- [ ] deer-flow 在 `http://localhost:8089/research/` 返回真实研究回复

qm 协作

- [ ] `docker compose -f compose.qm.yaml ps` 显示 qm 服务运行中
- [ ] `http://localhost:8182` 加载 qm Web UI
- [ ] 管理员可以登录
- [ ] qm 可以列出 Pacgate 工作流类别
- [ ] qm 可以通过桥接执行一个 Pacgate 工作流

Ollama

- [ ] `ollama list` 显示所需模型
- [ ] `ollama ps` 的 PROCESSOR 列显示 GPU，而不是 `100% CPU` - 运行 `scripts/test-ollama-gpu-residency.ps1`，退出码必须为 0
- [ ] deer-flow 可以调用 Ollama 进行推理
- [ ] qm 可以调用 Ollama 进行推理

> CPU-only 陷阱 - 在判定一台机器"慢"之前请先读这里。
>
> Ollama 只在服务启动时枚举一次计算设备，而它是在启动之后才把 ROCm/Vulkan 后端解包到 `lib\ollama\<vendor>_*` 的。若服务在后端解包完成前就已启动，它会把结论 定为"仅 CPU"，并在整个生命周期内保持该结论 - 安装完全正常、GPU 受支持，且没有任何 报错。
>
> 2026-11-04 在 RX 7600 XT 机器上的实测：
>
> 该机器所有本地模型在 CPU 上跑了 3.5 小时。一次"Reply with exactly: OK"耗时 377 秒；重启 Ollama 后同一请求耗时 7.4 秒，`100% GPU`。期间对话轮次看起来 像卡死，而 `smoke-full-stack` 仍报告全部绿灯。
>
> 若本地推理明显变慢或智能体看似停滞，第一个动作：
>
> ```
> powershell ollama ps # PROCESSOR ■■■ GPU■■■■ 100% CPU
> .\scripts\test-ollama-gpu-residency.ps1
> ```
>
> 若显示 CPU，请重启 Ollama 应用（不是重启模型）后复检。不要重装 ROCm、不要更换 模型、也不要改动技术栈 - 安装几乎肯定是正常的。
>
> 也正因如此，该项检查已纳入 `run-all-checks.ps1`：全新 AIPC 首次启动时可能触发同一 竞争条件。

数据

- [ ] `./data/tenants/` 目录存在且可写
- [ ] `./openviking/` 目录存在并在重启后持久化
- [ ] 通过 API 上传文档正常

- [ ] deer-flow 研究运行后事项记忆持久化
- [ ] 跨会话召回：通过 OpenViking `remember` 存储的事实可在后续会话中通过 `search` 召回

部署后管理栈

启动和停止

```
# ■■■■■
docker compose -f compose.prod.yaml up -d

# ■■■■■
docker compose -f compose.prod.yaml down

# ■■ qm
cd C:\pacgate-ai-pr\deploy\client-bundle\qm-pacgate
docker compose -f compose.qm.yaml up -d

# ■■ qm■■■ -v■■■■■■■■■
docker compose -f compose.qm.yaml down
```

> `qm up` / `qm down` 在 Windows 上无法运行（CLI 的 `which()` 直接调 `/bin/sh` → ENOENT）；compose 是 AIPC 上唯一可用的 qm 生命周期，
> 并且它承载了保证模型路由与沙箱链路持久的补丁。

更新到新版本

```
cd C:\pacgate-ai-pr\deploy\client-bundle
.\install.ps1 -Update
```

`-Update` 现在会自动完成以下全部工作，因此不再需要单独执行 `git pull`（那一步最容易被遗忘，而一旦遗忘，就会让新镜像配上旧配置静默运行）：

步骤	作用
1	刷新仓库工作区（仅快进 fast-forward）
2	从模板重新渲染 MCP 配置，变更前先备份
3	拉取新的 GHCR 镜像
4	重启技术栈，并重载 nginx

当机器上存在本地改动时，它会拒绝执行而不是猜测：

- 仓库有未提交改动 → 跳过仓库更新，并列出被改动的文件。不会 stash、 reset 或丢弃任何内容。
- 存在远端没有的本地提交 → 不合并、不变基。仓库保持原样，其余更新继续进行。

如需仅更新镜像，可使用 `-SkipRepoPull`：

```
.\install.ps1 -Update -SkipRepoPull
```

qm 只做报告，不自动更新

如果本机运行了 qm，`-Update` 还会检查其沙箱镜像是否仍与源码一致：

```
[OK] qm sandbox matches its source (38e062ec7c5c...)
```

或

```
[WARN] qm sandbox source has CHANGED since the image was pinned.
       qm is running OLD skills and tools. Rebuild + repin:
         cd deploy\qm-pacgate
         npm exec qm -- sandbox build   # ■■■■■■■ digest ■■■■
         pwsh -File ..\..\scripts\qm-sandbox-fingerprint.ps1 -Write
```

这一点很重要：qm 的智能体运行在一个按 digest 固定的沙箱镜像里（记录在 `deploy/qm-pacgate/qm.config.jsonc`）。按 digest 固定本身是正确的 — 隔离边界 就应当不可变 — 但这意味着仓库更新可以改动 `deploy/qm-pacgate/sandbox/`，而镜像完全不变。于是智能体继续使用旧的 skills 和 tools 运行，任何地方都不会报错。

`-Update` 对此只报告、不自动重建，因为重建需要 Node 24 + npm + buildx， 且之后必须重新固定 digest — 这属于配置变更，不应在客户机器上无人值守地执行。 一次错误的自动重建，比一条可见的警告更糟。

更新会保留数据：- `./data/tenants/`（卷挂载）— 事项、文档、记忆 - Postgres 数据（命名卷）— 元数据数据库

这台机器是不是最新版本？直接问它

```
curl.exe -s http://localhost:8089/version

{"version":"<release>","revision":"<git sha>"}
```

这里返回的是编译进正在运行的 pacgate-api 二进制文件的版本，以及它所基于的 commit。它故意不回显 compose 中的 pin 或镜像 tag：那些记录的是“曾部署了什么”，而真正要捕获的故障，恰恰是所部署的产物与实际运行的进程不一致。

在此之前，无法区分一台最新机器和一台落后的机器 — 从外部看两者完全相同， 唯一的办法是 SSH 进去读 compose 文件，并寄希望于容器与之相符。

安全修复之后：检查由旧版本清理过的文档

有些修复无法靠发布镜像来修复既有数据，0.1.31 的净化器改动就是如此。截至 0.1.30 （含）的版本会把 chunk 标记为 `sanitized`，却把原始文本留在行里，随后检索门控又把这段文本送给了对话。升级镜像不会重写这些行。

因此，凡运行过 0.1.28、0.1.29 或 0.1.30 的机器，更新后都应执行审计：

```
pwsh -File C:\pacgate-ai-pr\scripts\audit-sanitized-chunk-egress.ps1
```

该脚本只读，不会改动任何内容。机器干净时输出：

```
== RESULT: no raw Tier-1 match found in a sanitized chunk ==
```

其他结果会列出受影响的文档：

```
== RESULT: 106 raw match(es) inside SANITIZED chunks - LEAK PRESENT ==
affected documents (up to 25):
  matter=56723800-...  doc=820f4904-...  name=leak
```

对每个受影响的文档，重新执行其净化任务。该任务现在会用脱敏后的渲染文本替换陈旧的 chunk 内容，既修好该行，又保留检索覆盖。有一个脚本可以一次处理全部受影响的文档：

```
# ■■■■■■■■■■■ — ■■■■■■■■■■
pwsh -File C:\pacgate-ai-pr\scripts\remediate-sanitized-chunk-egress.ps1 -WhatIf

# ■■■■■
pwsh -File C:\pacgate-ai-pr\scripts\remediate-sanitized-chunk-egress.ps1
```

该脚本使用 `deploy\client-bundle\.env` 中的 `PACGATE_API_EMAIL` 与 `PACGATE_API_PASSWORD` 登录，因此需要真实凭据 —— 若密码仍是占位符，它会以退出码 2 明确报错，而不会半途而废。

如需逐个文档手动处理，等价命令如下：

```
$login = Invoke-RestMethod -Uri "http://localhost:8089/pacgate/api/auth/login" `
  -Method Post -ContentType 'application/json' `
  -Body '{"email":"admin@pacgate-law.com","password":"<■■>"}'
$hdr = @{ Authorization = "Bearer $($login.token)" }
Invoke-RestMethod -Uri "http://localhost:8089/pacgate/api/documents/<■■-id>/sanitize" `
  -Method Post -Headers $hdr -ContentType 'application/json' -Body '{"data_level":"T3"}'
```

随后再次运行审计，确认该文档不再出现。不要跳过这一步：修复脚本报告的是它做了什么，审计衡量的是事实如何；在第一台机器上二者曾出现分歧 —— 而且出现了两次 —— 正是由此发现了审计脚本自身的两个缺陷。

顺序很重要：必须先更新到 0.1.31。 在 0.1.30 及更早版本中，净化任务根本没有重写路径，重新执行不会带来任何改变，审计仍会列出全部受影响的文档。

开始前有两点值得了解：

- 该修复会重新生成文档的嵌入向量。 它会重写全部 chunk，因此大文档会按 chunk 数量产生嵌入调用。这是正确的，但代价与文档大小成正比，而非与实际泄漏量成正比。
- 直接删除这些行同样安全，且更快，但在重新净化之前会失去该文档的 RAG 覆盖。优先选择重新执行净化任务。

这两条路径在 0.1.30 及更早版本上都无效：那些代码根本没有重写路径，重新执行净化任务不会带来任何改变。请先更新版本。

切换模型

deer-flow（研究工作空间）：1. 编辑 `deer-flow-config.yaml` — 重新排序 `models` 列表（第一项 = 默认） 2. 重启：`docker compose -f compose.prod.yaml restart deer-flow`

qm（协作工作空间）：1. 若模型集变更，编辑 `deploy/client-bundle/qm-pacgate/qm.config.jsonc` 2. 重建核心（compose 保留补丁挂载，路由持久）：`powershell cd C:\pacgate-ai-pr\deploy\client-bundle\qm-pacgate docker compose -f compose.qm.yaml up -d --force-recreate core`

注册新用户

`/api/auth/register` 仅限首位用户，引导账号创建后一律返回 403（见 Stage 3）。此后的每个账号通过管理员路由创建：

```
# /pacgate ■■■■—■ Stage 3 ■■■■Bearer = ■■■■■
# (■ POST /pacgate/api/auth/login ■■■■ Stage 3 ■■■■)
$body = @{email="<user>@pacgate-law.com"; password="<password>"} | ConvertTo-Json
Invoke-RestMethod -Uri "http://localhost:8089/pacgate/api/auth/users" -Method Post `
  -Headers @{Authorization="Bearer <admin-token>"} -Body $body -ContentType "application/json"
```

备份数据库

```
docker exec pacgate-db pg_dump -U pacgate pacgate > backup.sql
```

查看日志

```
docker compose -f compose.prod.yaml logs -f pacgate-api
docker compose -f compose.prod.yaml logs -f deer-flow
```

已知限制

- `qm up` / `qm down` 在 Windows 上无法运行。 qm CLI 的 docker 生命周期 只有 POSIX 一条路：它的 `which()` 执行 `execFileSync("/bin/sh", …)`，在每台 Windows AIPC 上都是 `ENOENT`（2026-10-06 实测）。请使用上面的 compose 路径 （`docker compose -f compose.qm.yaml up -d`）——同样的拓扑与容器名，并且 承载 Pacgate 补丁，重建时不会丢失任何东西。
- 每台机器都有自己的独立 Postgres 和 `./data/tenants/` 目录。除非你之后添加私有网格和同步或单一权威模型，否则事项数据不会在机器之间共享。

- PkuLaw 连接器令牌已过期。在 `https://mcp.pkulaw.com` 重新生成，并在试点期间需要中国法律搜索时在 `.env` 中设置 `PKULAW_API_KEY`。
- 四个 WASM crate（citation-check、clause-parser、doc-validator、rule-engine）仍是桩。这些是未来蓝图工作，不影响 Phase 1 试点功能。
- 模型选择： API 默认使用目标机器上可能不存在的模型。Stage 3 之后，应用 按租户的模型覆盖，使 LLM 层级指向该机器 `ollama list` 中实际存在的模型。 推荐的试点集（2026-08-28 基准测试）：`gemma4:12b-it-qat`（Main — 13 秒/工具轮，模式有效的工具调用，端到端验证）、`qwen3.8:27b-mtp-q4_K_M`（Mid — 73 秒/工具轮，批量表格审查质量更强）、`nomic-embed-text:latest`（嵌入）。交互层级避免使用推理模式模型（例如 nemotron）——它们可能挂起长 docx 生成。SQL 模板见 `plans/007-aipc-full-installation-handoff.md` 附录 A。

引用的文件

文件	用途
deploy/client-bundle/compose.prod.yaml	pacgate-api + deer-flow + Postgres + nginx 的 Docker Compose
deploy/client-bundle/install.ps1	核心栈的一键 Windows 安装程序
deploy/client-bundle/setup-qm.ps1	qm 引导脚本（密钥、配置、沙箱构建）
deploy/client-bundle/.env.example	客户端密钥模板
deploy/client-bundle/ollama-models.txt	要预拉取的模型
deploy/client-bundle/deer-flow-config.yaml	deer-flow 多模型配置（5 个模型，可切换）
deploy/qm-pacgate/qm.config.jsonc	qm 本地部署配置
deploy/SETUP-AND-OPERATIONS.md	完整的 3 天现场安装指南（参考）
deploy/DEPLOYMENT-GUIDE.md	工程师级部署细节（参考）